IoT Final Report

Greenhouse Mesh Network

Relay Node & Node-RED Co-Design

Student: Vincenzo Cuzzocrea

Course: Internet of Things [E060]

Academic Year 2025/2026

Semester: Summer



POLITECHNIKA
LUBELSKA
WYDZIAŁ ELEKTROTECHNIKI
I INFORMATYKI

1. Introduction

The Internet of Things (IoT) is reshaping automation by enabling low cost, networked embedded devices to sense the physical world and actuate responses in near real time. This project applies those principles to a smart greenhouse hypothetical scenario: a mesh of heterogeneous nodes (sensors, actuators and a central coordinator) collaborates to monitor and regulate the growing environment autonomously.

Each node in the mesh serves a well-defined role. Sensors continuously sample physical quantities (soil moisture, ambient light, barometric pressure, air temperature and humidity, motion, door state and heart rate) and publish structured JSON packets over MQTT. Actuators (a relay-controlled water pump, two OLED displays and an SD-card data logger) subscribe to the relevant topics and respond to commands issued either automatically by logic running on the Raspberry Pi or (in future extensions) by a human operator through a dashboard.

This report focuses on two specific contributions made within the group project:

- Design, implementation and deployment of the relay node: a Wemos D1 Mini (ESP8266) that controls a 5V relay module connected to a water pump.
- Co-design of the Node-RED flows running on the Raspberry Pi broker, which encode the automation logic that bridges every sensor with every actuator.

The sections that follow describe the overall system architecture, detail the firmware written for the Wemos, analyse each Node-RED flow, and draw conclusions from the experience.

---

2. System Overview

The greenhouse mesh is built around a Raspberry Pi that runs Mosquitto as the MQTT broker and Node-RED as the automation engine. All nodes connect to the same Wi-Fi network and communicate exclusively through MQTT, keeping the architecture decoupled: any node may be replaced or upgraded without modifying the others.

Student	Cuzzocrea
Course	Internet of Things
Role	Relay Node Developer & Node-RED Co-Designer
Hardware	Wemos D1 Mini (ESP8266) + 5V Relay Module
Broker	Raspberry Pi | Mosquitto + Node-RED
Protocol	MQTT over Wi-Fi

The full topic namespace used by the mesh is summarised in the table below.

MQTT Topic	Description
greenhouse/sensor/soil	Soil moisture sensor publishes readings
greenhouse/sensor/light	Light sensor (lux) publishes readings
greenhouse/sensor/barometric	Barometric pressure & temperature sensor
greenhouse/sensor/temp_and_humidity	DHT-type humidity sensor
greenhouse/sensor/button1	Physical button triggers OLED summary
greenhouse/sensor/heartrate	Heart-rate monitor (BPM)
greenhouse/security/motion	PIR motion detector
greenhouse/security/door	Door open/close sensor
greenhouse/actuator/relay	Command topic — Wemos D1 Mini listens here
greenhouse/actuator/relay/status	Status feedback published by Wemos
greenhouse/actuator/oled1	OLED display 1 (environment summary)
greenhouse/actuator/oled2	OLED display 2 (alerts & emergencies)
sd_logger/save	SD-card logger receives formatted log lines

A clear naming convention separates sensor data (greenhouse/sensor/*), actuator commands (greenhouse/actuator/*) and security events (greenhouse/security/*). The Raspberry Pi broker sits at the centre: it receives every sensor reading and - after evaluating the logic in Node-RED - publishes commands to the appropriate actuator topics. The relay node developed for this project listens on greenhouse/actuator/relay and confirms every state change on greenhouse/actuator/relay/status.

3. Wemos D1 Mini Firmware

3.1 Hardware Setup

The Wemos D1 Mini is a compact Wi-Fi development board based on the ESP8266 SoC. It is paired with a single-channel 5V relay module whose control input is wired to digital pin D1. The relay switches the mains circuit of a small water pump: when the relay coil is energised (HIGH signal on D1) the pump runs; when it is de-energised (LOW) the pump stops.

At boot the firmware drives D1 LOW, ensuring the pump starts in a safe, off state regardless of the previous relay position.

3.2 Libraries

- ESP8266WiFi - hardware Wi-Fi driver for the ESP8266.
- PubSubClient - lightweight MQTT client; handles publish, subscribe, and the keep-alive loop.
- ArduinoJson - efficient JSON serialization/deserialization with a stack-allocated document model.

- NTPClient + WiFiUDP - retrieves the current Unix timestamp from pool.ntp.org so that every status message carries accurate timing information.

3.3 Message Format

The group agreed on a shared JSON dataframe for all devices proposed by me. Every packet carries four mandatory top-level fields and a data object whose contents differ by device type:

```
// Inbound command (broker → relay node)
{ "action": "ON" }     // or "OFF"


// Outbound feedback  (relay node → broker)
{
  "device_id":   "cuzzocrea",
  "device_type": "actuator",
  "timestamp":   1716800000,   // Unix epoch from NTP
  "action":      "ON",         // reflects executed action
  "data":        {}
}
```

Keeping the inbound command minimal (only the action field is parsed) makes the node resilient: it ignores any additional metadata a future version of the broker might attach.

3.4 Annotated Source Code

The complete firmware is listed below with inline explanations grouped by functional block.

- Includes and configuration constants

```
#include <ESP8266WiFi.h>
#include <PubSubClient.h>
#include <ArduinoJson.h>
#include <NTPClient.h>
#include <WiFiUdp.h>

//wifi and broker info
const char* ssid = "Nothing Phone 2a";
const char* pass = "*********";

const char* mqtt_server = "10.180.220.29";

//topic infos
const char* listen_topic = "greenhouse/actuator/relay";
const char* write_topic  = "greenhouse/actuator/relay/status";

WiFiClient espClient;
PubSubClient client(espClient);

const int relayPin = D1;
```

```
//network clock for timestamp
WiFiUDP ntpUDP;
NTPClient timeClient(ntpUDP, "pool.ntp.org", 0, 60000); //Unix timestamp
```

The MQTT server IP is an address dynamically assigned to the Raspberry Pi on the shared hotspot (it could change in future connections). QoS for subscriptions is handled by PubSubClient at QoS 0 by default; the broker-side subscriptions in Node-RED use QoS 2 for guaranteed delivery of sensor data.

- sendStatus(): JSON feedback publisher

```
//function to publish json feedback on write_topic
void sendStatus(String relayStatus) {
  //create json doc in memory
  StaticJsonDocument<256> doc;

  //compile packet as chose from dataframe
  doc["device_id"] = "cuzzocrea";
  doc["device_type"] = "actuator";
  doc["timestamp"] = timeClient.getEpochTime(); //gets timestamp from NTP
  doc["action"] = relayStatus;
  //create nested obj "data" empty
  JsonObject data = doc.createNestedObject("data");

  //transform obj in string
  char buff[256];
  serializeJson(doc, buff);

  //send message
  client.publish(write_topic, buff);
  Serial.println("Sent JSON feedback: ");
  Serial.println(buff);
}
```

A StaticJsonDocument is allocated entirely on the stack (256 bytes), avoiding heap fragmentation. Important on a device with only 80 KB of usable RAM. The NTP timestamp makes log correlation possible when messages from multiple nodes are stored by the data logger.

- callback(): MQTT message handler

```
//callback function for receiving
void callback(char* topic, byte* payload, unsigned int length){
  Serial.print("\nMessage arrived on topic: ");
  Serial.println(topic);

  //create json obj to read message
  StaticJsonDocument<256> doc;
  //try to decode message
  DeserializationError error = deserializeJson(doc, payload, length);
  //check error
```

```
    if(error){
      Serial.print("Error: message is not a valid JSON! ");
      Serial.println(error.c_str());
      return; //interrupt if its not a json
    }

    //only extract "action" from json
    const char* action = doc["action"];

    if(String(action) == "ON"){
      digitalWrite(relayPin, HIGH);
      Serial.println("Relay turned ON via JSON");
      sendStatus("ON");
    } else if(String(action) == "OFF"){
      digitalWrite(relayPin, LOW);
      Serial.println("Relay turned OFF via JSON");
      sendStatus("OFF");
    }
  }
```

The callback validates the incoming payload as JSON before touching any hardware. This prevents undefined behaviour if another node accidentally publishes a malformed message to the relay topic. Only the action field is extracted, keeping the handler forward compatible with richer command envelopes.

- reconnect(): resilient broker connection

```
  //function to connect to broker
  void reconnect(){
    while(!client.connected()){
      Serial.print("Trying MQTT connection...");
      String clientId = "WemosClient-";
      clientId += String(random(0xffff), HEX); //give a random id number

      Serial.print("\nRandom ClientID set: ");
      Serial.println(clientId);

      if(client.connect(clientId.c_str())){
        Serial.println(" Connected to broker!");
        client.subscribe(listen_topic); //subs to chosen topic
      } else {
        Serial.print("Failed. Status=");
        Serial.print(client.state());
        Serial.println(" Retry in 5 seconds...");
        delay(5000);
      }
    }
  }
```

A random suffix is appended to the client ID on every connection attempt. This avoids the 'client ID already in use' rejection that would occur if the device reboots while the broker still holds the previous session.

- setup() and loop()

```
void setup(){
  Serial.begin(115200);
  pinMode(relayPin, OUTPUT);
  digitalWrite(relayPin, LOW); //reset relay

  //start wifi connection
  WiFi.begin(ssid, pass);
  while(WiFi.status() != WL_CONNECTED){
    delay(500);
    Serial.print(".");
  }
  Serial.println("\nWiFi connected");

  //start clock client
  timeClient.begin();

  client.setServer(mqtt_server, 1883);
  client.setCallback(callback);
}

void loop(){
  //constantly update clock in background
  timeClient.update();

  if(!client.connected()){
    reconnect();
  }
  client.loop();
}
```

The loop is intentionally minimal: timeClient.update() keeps the NTP clock synchronised in the background, and client.loop() processes incoming MQTT packets. All business logic lives in the callback, which is invoked by PubSubClient inside client.loop() when a matching message arrives.

---

4. Node-RED Flows on the Raspberry Pi

The Raspberry Pi runs six Node-RED flow tabs, each responsible for a distinct concern. The design principle was separation of responsibilities: global state management, automation

logic, security, persistence, monitoring, and health alerting are handled in dedicated tabs rather than in one monolithic flow. This makes the system easier to extend and debug.

4.1 Global & Datalogging

To prevent redundant requests and maintain a clean network traffic, the system utilizes an approach based on global variables. By subscribing to the sensor topics (greenhouse/sensor/#), the server extracts data fields from the JSON payloads and stores them in the system's RAM (e.g. light, pressure, temperature, humidity). Any downstream process can access this data instantly.

Concurrently, a logging flow captures all incoming messages, converts them into human-readable strings, and routes them to the SD storage node (sd_logger/save).

4.2 Irrigation Automation & Security Override

Water pump management is based on a combined logic with hysteresis, designed to prevent stress on both the hardware and the plants:

- Activation: Occurs only if soil moisture is at a critical level (< 30%) and sunlight is indirect (< 2000 lux, to prevent leaf scorch).
- Deactivation: Triggers when optimal soil moisture saturation is reached (> 80%).

This automation is overlaid with a security (intrusion detection) level. Motion and door opening sensors communicate directly with the irrigation block: if a presence is detected inside the greenhouse while the pump is running, the system automatically dispatches a shutdown command (override) to avoid spraying the operators, concurrently triggering a visual alert.

4.3 HMI Interfaces & Medical Monitoring

Leveraging the same MQTT infrastructure, the server manages human-machine interaction via two distributed OLED displays:

- On-Demand Environmental Monitoring: By pressing a physical push-button (greenhouse/sensor/button1), the first display retrieves the global variables, providing the operator with an instant dashboard of current values (Temperature, Humidity, etc.) without saturating the network with continuous updates.
- Medical Emergency (Heartbeat Monitor): The second display is dedicated to occupational safety. The server analyzes data from a heart rate monitor worn by the operator. If critical anomalies are detected (bradycardia under 40 BPM or tachycardia over 130 BPM), the system immediately publishes a "MEDICAL EMERGENCY" payload. This demonstrates how an IoT network can seamlessly integrate agricultural automation with personnel safety monitoring.

5. Conclusions

The project successfully realised a working IoT mesh in which sensors and actuators cooperate through an MQTT broker hosted on a Raspberry Pi. The relay node developed for this work demonstrates a complete embedded software stack: Wi-Fi connectivity, MQTT publish/subscribe, JSON-based messaging, NTP timestamping and resilient broker reconnection. All within the constraints of an ESP8266 microcontroller.

The Node-RED flows provide a clear separation of concerns across six tabs: global state, irrigation automation, security, data logging, on-demand display and medical alerting. The dual-condition irrigation logic (soil moisture + ambient light) and the hysteresis thresholds show how simple rules can prevent hardware damage and plant stress without complex algorithms.

Several lessons emerged from the experience:

- A shared JSON data frame agreed upon by the whole group was essential: it allowed nodes developed independently to interoperate without integration surprises.
- Global context variables in Node-RED are a practical way to share state across flows without introducing additional MQTT round trips.
- Resilient reconnection logic on the embedded side (random client ID, retry loop) proved necessary in a shared Wi-Fi environment with intermittent connectivity.
- The security and medical alert flows highlighted how a sensor mesh can be extended to serve safety critical purposes beyond its primary environmental function.

Possible future improvements include TLS-encrypted MQTT connections, retained messages for state recovery after broker restarts, a Node-RED dashboard UI for remote monitoring and OTA (Over-the-Air) firmware updates for the Wemos nodes.

---

6. References

- Espressif Systems. ESP8266 Technical Reference, v1.7. 2020.
- Eclipse Foundation. Mosquitto MQTT Broker. https://mosquitto.org
- Node-RED Foundation. Node-RED Documentation. https://nodered.org/docs
- Benoit Blanchon. ArduinoJson Library v6. https://arduinojson.org
- Nick O'Leary. PubSubClient for Arduino. https://pubsubclient.knolleary.net
- IETF RFC 9431. MQTT Version 5.0. 2023.